

Okay, okay, all right. Okay, I just want to start by thanking uh all the organizers for this opportunity to present today. Uh I'm very excited to talk to you about uh how we've been using deep learning models to uh to understand regulatory DNA. Um and I'm gonna sort of focus a talk on uh uh uh a bit of benchmarking, but also a bit on you know showcasing why understanding what models actually learn is very an important part of benchmarking and how it can give you insights into designing better critical models and and better uh better benchmarks as well. So just to focus the talk, I'll be referring to the process of transcription. As you know, genes are transcribed to RNA, translated to proteins, and this process of transcription is regulated in cis by a number of these regulatory elements embedded in the genome, you know, like promoters, enhancers, insulators. And these elements are typically recognized and bound by proteins, like transcription factors. They will often... Okay, okay, sorry. Okay, I just want to start by thanking all the organizers for this opportunity to present today. I'm very excited to talk to you about how we've been using deep learning models to understand regulatory DNA. And I'm going to sort of focus a talk on a bit of benchmarking, but also a bit on showcasing why understanding what models actually learned is very an important part of benchmarking and how it can give you insights into designing better critical models and better benchmarks as well. So just to focus the talk, I'll be referring to the process of transcription. As you know, genes are transcribed to RNA, translated to proteins. And this process of transcription is regulated in cis by a number of these regulatory elements embedded in the genome, like promoters, enhancers, insulators. And these elements are typically recognized and bound by proteins like transcription factors. They will often tend to bind. Yeah, I wonder if you can get rid of the... Okay. So these proteins typically tend to recognize and bind sequence-specific motifs. And they bind in complex configurations. So there is not just an additive effect of things binding together, but there's often complex or combinatorial synergistic effect of proteins binding DNA. And understanding this kind of cis-regulatory code, that is a sequence syntax of these elements, is a pretty fundamental question in gene regulation. Now, the nice thing is you can actually use a variety of different profiling assays to capture different biochemical markers that represent different kinds of regulatory activity. For example, this track here is showing a ChIP-seq experiment capturing binding of this black triangular protein at specific locations. Similarly, you have various kinds of epigenetic or chromatin modifications. Those can also be captured by various readouts and, of course, measure DNA methylation, gene expression, and so forth. So we can start using large-scale genome profiling in a variety of different cell types and tissues to create these reference regulomes. Just as we have the reference genome, we now have large numbers of assays performed across the entire genome in hundreds and thousands of cell types, tissues, cell lines, and now at single cell resolution. And one important point here is that a lot of this activity of the genome is highly context-specific. So here's a piece of the genome, one particular gene. These are all a variety of different immune cell types. We're looking at chromatin accessibility, which is sort of a measure of open chromatin, where proteins typically bind in

the genome. And you can see, you know, like different parts of the genome, like this promoter idea is open in specific cell types and not open in others. Others are open in other cell types. So context really matters. And that's a really important piece of this puzzle as well. So I'll start off with models that we've built in the lab and literally the opposite of a foundation model. These are models trained on individual experiments. So you performed a ChIP-seq experiment or a DNA's or a tax-seek experiment in some cell type. Your goal is to precisely capture that model in the context of how DNA sequence regulates that particular assay's activity. So it's a model per experiment. It's a sequence-to-function model, which means it takes DNA sequence as the input and maps it to base pair resolution profiles, coverage profiles of an experiment. And these are very simple models. They only take about two kilobases of sequence at a time. So they are short receptor fields. But they're really modeling data at single base pair resolution. That's a key aspect. And we built a variety of these models. We first published BPNet in 2021. This was focused on transcription factor binding, so protein DNA interactions for chip-exo, chip-seq, cut and run-like assays. Then we released Chrome BPNet, which is sort of a version of this for attack-seq, DNA-seq, single-cell attack-seq. We've also recently released ProCapNet, which is a similar model for nascent and steady-state transcription start site activity. So that's for ProCap, cage, and rampage-like assays. And we've been expanding across the assay space. So we have a specialized model for every assay, kind of the exact opposite of a classical foundation model. And the reason we did it this way is because we felt like you can go pretty far, firstly, with small models. You have no control of how you train them, and we can interpret them at scale very robustly to understand what they're learning, thereby providing very strong baselines for much larger models. That's the idea behind it. So one of the first things we can do with a trained model, if assuming it's accurate, is, for example, we can take, I'm sorry, you can't see this very well with the stuff on top. What you're seeing here is actually sequence, and the model has been already trained to predict profiles. We're able to reverse engineer what the model is learning for a specific sequence of interest and get these contribution scores. Every base pair in the sequence is given a score. That score highlights the contribution of that base pair to the predicted activity from the model. So it allows you to exactly identify base pairs, motifs, and other features in the sequence that are likely driving activity. And we've developed a variety of these kinds of approaches. I'll walk you through a few examples of how this works. So here's the MIC promoter in the human genome. There are two transcription start sites. We're looking at a Luke-Mes aligned K562. Here's the experiment. It's a pro-CAP experiment. It gives you base pair resolution precision of where transcription start sites are, and it measures nascent transcription. You train a model, this kind of small, you know, pro-CAP-ed model on that data. It really beautifully captures the data very well. So this is a cross-validation chromosome. So this sequence has not been seen in training. You can see the model very nicely recapitulates what was seen in the data. The question then you might want to ask is like, how is the model making this prediction? That's the interesting question. Because you already have

the data. There's not much value in predicting data you already have, unless the model is actually learning some very exciting biology. So we can take the model, we can apply these post hoc attribution methods that take the model as a lens, and you can now take these base pairs, which when you just look at them naively, you might not know what they do, and use a model to figure out which base pairs are interesting. So when you do that, you get this contribution score profile, which says most base pairs in the sequence are not doing much. But right there, there is a really interesting syntax that pops up. At this main transcription side of MIC, there's a classic TAINR. It's initiator motif. The model picks it up very nicely. There's a cutter box exactly 30 base pairs upstream and an SPBRE motif further upstream. For this other TSS right here, again, you see the cutter box and the SPI motif, but watch this one, right? You see how the model focuses on a single A. It says that there's a single A in the genome that allows you to initialize transcription. Obviously, there are a million A's or billion A's inside the human genome. Every A doesn't initiate transcription. The reason this happens is because this A is exactly 30-base pairs downstream of a start box, right? And I don't have time to show you this, but you can do ablation experiments where you can mutate one motif, mutate another one, and see the effect on the activity. And you can clearly see that if you mutate, for example, or you have a mutation in the start box, this INR disappears. So you can see how the model beautifully learns epistasis, but also shows you the ability of using the model to get really beautiful, clean insights into the actual sequence code that drives transcription. Now let's say we switch to a different assay. So it's the same cell type. It's the same promoter. So this time we switch to DNA-seq, which measures chromatin accessibility. It's the exact same sequence in the same cell type. Now we take a different model. This one is called ChromEPI, not ProCapNet. This one is trained on DNA-seq data. And you can see the model very nicely predicts the data as seen at base-pair resolution. I'll talk about bias correction in a bit, but one very important thing to realize is, and I'm sure we all know this, but often don't really process it, is that the data that you see from an experiment is not the ground truth. It is a ground truth confounded by various technical artifacts. So if you train a very powerful model on confounded data, your model is going to learn biology and confounding. That's what we see with many of these assays. For example, ChromBPAT, if you train any sequence to function model, if you train it on ATAC-seq or DNA-seq, it hyper-focuses not only on the regulatory biology, but also on the enzyme biases. And I'll show you the massive impact that these can have on what you see from the experiments. But anyway, the model predicts it very well. We can do bias correction. Just take my word for it right now. And now you take the same promoter and interpret it through the lens of chromatin accessibility, right, not nascent transcription, you see a very different code pop up. It's the same sequence. The contribution scores are very different. These INR start boxes and so forth don't pop up. But you see these kind of GC-rich sequences pop up. And there is a GATTA motif. It's the reverse complement of GATTA. And you can see how it specifically pops up in the chromatin accessibility track, and it's hugging this particular initiator, right? So you have different parts of the sequence

being utilized by different biochemical processes. You have the initiator, the tarot box really positioning transcription, but the chromatin accessibility is regulated by a completely different syntax in the exact same sequence. And the model can tell you that. You can decompose this further because luckily in this cell line, we have ChIP-seq data for multiple transcription factors. So we can also train models on the ChIP-seq data itself, one model for every transcription factor, and interpret the same sequence through the lens of those models. And you see that the SP1 ChIP-seq, right, the model that's trained on that, its contribution scores almost exactly mimic the chromatin accessibility contribution scores, except for the GATA motif. Because SP1 does not recognize GATDA. It can recognize these GC-rich-like sequences. But if you look at a GATA ChIP-seq model trained in the same cell line, it precisely picks up the GATA motif, right? So you can now say that the chromatin accessibility track that you observed over MIC is essentially a combination of GATA and SP1 collaborating to open chromatin. Thereafter, you have initiation which is positioned by these specific initiator modifies. Now let's switch the context again. This time we're going to keep the MIC promoter as it is. We're going to keep the assay again DNA-seq. We're going to switch the cell type. We're going to go from a K562 leukemia cell line to a HepG2 liver carcinoma cell line. And if you look at the data, right, just look at the observed data, it looks almost identical. You can't really make out the difference between the two cell lines, at least base by resolution. The model predicts it almost identically as well, does bias correction. It's very unfortunate that it's hiding the last track. But luckily, it's not hidden the most important piece, which is, take my word for it, that the contribution scores from the HepG2 model look exactly like the K562 model, except at this position. You see this GADA motif right here? It's gone. It's one motif. That's about it. And the model has never seen gene expression. It's just training on proton accessibility using sequence only. The GADA transcription factor is not expressed in HepG2. So the model has somehow learned, not magically, but looking at other sequences that are accessible, that despite the profile looking nearly identical, the GADA motif is not actually regulating accessibility in the liver saline. And what effect does that have? Chromatin accessibility is significantly lower in HEPG2 than K562. So it affects the magnitude of accessibility, but not the profile shape. So there are different axes to these assays as well. We can go to an enhancer. This is an enhancer of MIC, validated. And we're going to look at DNAs. Again, observed, predicted. And this is the bias-corrected DNAs. We have a method to actually correct for the enzyme bias, which we'll use model interpretation itself to learn. We can interpret this enhancer, but let's look at ATTC-Seq. So these, again, DNA-seq and ATT-Seq are supposed to be assays capturing the same phenomenon of protein accessibility. But when you look at them at base-pair resolution, they look remarkably different. They literally don't look like the same thing at all, right? Despite they're supposed to capture the same thing. If you naively train a machine learning model on either of these assays, it beautifully mimics the assay itself. Look at our predictions. Doesn't it look exactly like ATT ?? C-Seq, and this one looks exactly like DNA-seq. But now if you correct the enzyme biases, which again, you

have a neural network learn the enzyme biases from the data itself, you can regress it out and look at the underlying latent, hopefully ground truth. And what you see for ATT ?? CK is after you correct for TN5 enzyme biases, you get this latent profile, which is very similar to the DNA's one. So the observed data that you got from the ATTAC-seq data was actually corrupted by this TN5 bias. It is not reality. It's not ground truth. It is a ground truth corrupted by a heavy enzyme bias. DNA's has a much less enzyme bias. So even after bias correction, the predictions look much more similar to the observed data. Now the cool thing is you can interpret both models. Again, two separate models trained on two different assays independently, never talking to each other. And you can ask the models, hey, tell me what syntax is driving activity accessibility in this enhancer. And you can see the two models give you nearly identical syntax. So after correcting biases, you really make them look nearly identical, not only in the predicted space, but in the most important space, which is actually the features being used to make predictions. And then you can take models trained on a variety of different other assays to support what you see and to understand the context specificity. So if you train a model on an MPRA, it gives you similar syntax, but not identical. For example, this motif here is specifically found driving activation, but not opening chromatin. So you can really start now dissecting the precise roles of individual base pairs of motifs and driving different biochemical processes through the process of transcription. Now these are all cool stories, but you need validation. So this is after all a benchmarking talk. So what we did here is we collaborated with Axel Whistle, Len Perakio and Michael Koziki, and they did this insane experiment where they took a handful of enhancers and they put these in mouse embryos. And these are in vivo reporters. So you're taking these enhancers, you're taking the sequences, you're mutating 12 base map blocks at a time, and they do random scrambles. So you don't know what exactly they're mutating. And you can put these into the embryo and then different parts of the embryo light up. So at the same time, you can figure out which cell type, for example, the enhancer is having an effect and what its effect size is. It's not super quantitative, but it tells you at least binary, you know, does it have an effect, does it have a strong effect, weak effect, and you can see where in the mouse embryo it has an effect, at what stage of development. We can use this as a ground truth to try to test whether models trained, this time, our models are trained on chromatin accessibility in a large number of human cell types and cell cell states. We're going to test them against a report of gene expression in a mouse. So these human models trained on accessibility is a very hard generalization problem. We get very nice predictive power. So like our models are able to get 88% specificity at 60% sensitivity. Now this is not 100%. It should not be because we are switching assay and we're switching species. But the most exciting piece is we can look at these blocks that cause effects and we can actually use the model to interpret what is causing the effect. So we can tell, for example, this gain of function effect is because the scramble destroyed a SNI repressor. This loss of function effect is because you scramble a SOX motif. So you can get really nice explanatory power to do this as well. Another

interesting application of this is to use this for variant effect prediction. So you can take any genetic variant, you can take a trained model, you can insert the reference allele, alternate allele, and make predictions, but also actually interpret the model and make interesting insights. For example, this C reference allele doesn't do much. There's a weak motif right here. And the G allele creates a really strong SPI motif that interacts with this hidden SPI motif. And they collaborate to create a massive amplification of the accessibility at that location. So you're again a nice way of interpreting genetic variants. We can do this at scale. So here we're trying to again use a benchmarking data set. Here what we're going to do is train a reference model on a lymphoblastoid saline from a European individual. It's a single model trained on the reference genome in a reference individual. We're going to use it to try to predict QTLs. These are genetic variants that have associations with changes in chromatin accessibility across a large panel of 100 diverse African individuals. So we're switching ancestry and we're using this QTL data as a ground truth. We train our model on a reference individual and then we'll make predictions for all these variants. On the x-axis is the actual observed data, the effect sizes. On the y-axis are the predicted. And you can see it's not perfect, but there's very nice sign concordance, about 0.69 correlation. This is one model trained on one data set on a reference individual. Now let's switch to the foundation models, right? The ones that are trained on large number of individuals, large number of data sets simultaneously in multitask mode. And we'll look at Informa, which was previous state-of-the-art. This new alpha genome that just came out a few weeks ago. On the y-axis here is again the correlation between predicted and observed effect sizes. The blue triangles you see here are chrome VPNet. That's a small model. On the x-axis, which you cannot see, are various thresholds for how you call significant QTLs. So we're changing the threshold just to be threshold independent. And we're going to see how performance of these models changes as a function of threshold. You can see that they perform pretty uniformly across threshold, which is nice. But you'll see that these, you know, the yellow points are using Enfirma as published in the original paper. And they perform really quite poorly. Part of the reason is because they have a weird strategy of predicting variant effects, which is they compute the effect of the variant on a 100 kilobase window of profile. So even though the variant has a local effect on signal, it gets washed out when you look at it 100 kilobases. If you fix that kind of bug, they do quite a bit better. That's the green points. But you can see the RL models actually outperform it by a decent margin. You can switch this prediction task to discriminating between significant QTLs and background QTLs, not just predicting effect size. The y-axis here is now average precision. Again, you're discriminating between variants that have effects and variants that don't. You can again see this is the baseline performance of the random classifier. This is NFOMA as published. This is NFOMA after correcting the variant effect bias. This is our Chrome BVNET models. What's really interesting is we are very much on par with them. But as soon as you increase the sequencing depth of the assay, not the size of the model, not the number of parameters, not the amount of data, just the reads, the number of reads, the

quality of the data, you see this giant boost in performance. So data quality, improving that often will boost your model better than simply increasing the size of your model or the number of parameters or the amount of data you have. And just to be clear, Enformer has two orders of magnitude more parameters than ours. It also is trained on 6,000 or 10,000 tracks simultaneously. Chrome unit is trained on a single track. It only has a 2KB receptor field versus Enformer's 100 kilobase receptor field. So despite having all the necessary advantages, capacity, model capacity, size, it ends up doing comparably or slightly worse than what we're able to do. Let's move to another class of foundation models, which is the self-supervised foundation models. These have been very big as well. They're often called DNA language models or regulatory language models. I'm going to focus here on what I'm referring to as annotation agnostic models. So these are models that are trained on entire genomes across multiple species in a self-supervised way to reconstruct sequences that are used as inputs. You just mask or you do autoregression. And the point is that they're supposed to be able to learn powerful representations, which you can then use for fine-tuning or probing for other supervised tasks. So we're looking at a few of the state-of-the-art models, and Evo2 can also be added to this bar. We're looking at DNA BERT, GNGNILM, HINAR DNA, nuclear transformer. We're going to test these models in the same variant prediction task that I showed you in the previous slide. Chrome DPNet, can it predict these covenant QTLs? And just for reference, AUROC 0.5 is random. Chrome DPNet's performance on two separate data sets is about 0.75 year and about 0.93R. So that's the baseline. The baseline is not a random predictor. The baseline is not another language model. The baseline is what should be the state-of-the-art small model that can get you the best performance. That's how you should be benchmarking. The trick that's used in all the papers is we compare to the previous iteration of what someone else did in the same space and see if it does better. Now the point is that in ZeroShot, all these models are basically random. They cannot predict these variants at all. You can probe them, which is you can add a linear head on top of the last layer of the model. They just barely cross random. Even if you fine-tune the whole model on the exact data that Chrome VPNet has access to, they're still substantially worse, right? So they're nowhere close. And you can add other DNA language models here too. It's the same story. So just because you have large parameter space and a lot of data doesn't necessarily mean your model is actually going to do well. In fact, a lot of these DNA language models that are trained often do very poorly on counterfactual prediction. Even if they're fine-tuned, they end up doing pretty poorly on these, especially on the non-coding genome. They're pretty good on coding sequence. A non-coding sequence, they really aren't. We have various other benchmarks too. We're looking at effects on the, these are in-genome effects with CRISPR experiments on the promoter of a gene called PPIF. Again, Chrome BPNAD, even though it's trained on chromatin accessibility, is able to predict effects on expression with like 0.65 correlation. If you take Nforma's expression head and you make predictions, it gets a correlation of 0.42. If you take Nforma's accessibility head and predict expression effects, it gets 0.62. So again,

the small model is able to outperform the big one. On this particular task, Chrome BPNET also outperforms AlphaGenome, the latest iteration of these, of this kind of models. You can go to the MPRA benchmark. So this is the KG5 competition that was held a few years ago. These are report assays where you do saturated mutagenesis of promoters and enhancers. Each point here is a promoter or enhancer of a specific gene. That's the name you're seeing. On the y-axis is the correlation of NFOMA's predictions against the ground truth. On the x-axis is the predictions of Chrome BPNET against the ground truth. The diagonal would be, they're equally performing. And if you're on the bottom of the diagonal, Chrome BPNET outperforms Enformer. On the top of the diagonal, Enformer would outperform Chrome BPNET. You can see right here, they're pretty much on par. And on some examples, we do substantially better. So overall, the key point is that when you're benchmarking these models, you need to kind of really think about what's your actual baseline, right? Five minutes. It cannot be, again, like I said, a random classifier or something like that. You have to push your baseline models to the limit. And in many cases, these small models trained on small amounts of data in specific ways can take you pretty far, besides having the advantages of interpretability. One last thing I want to show is that these large multitask models, like Informa, Deep C, Alpha Genome, again, they take sequences input and they're used to predict thousands of tracks simultaneously. There's another problem that occurs is since they are learning joint representations, the causality of these models starts coming into question. I'll show you a cool example of this. So here we're taking the deep C model, which is trained on, again, thousands of tracks. We're taking the reference sequence and we're interpreting it. You can see right here, there's a bunch of AP1 motifs that drive activity. This makes sense. AP1 is a primary transcription factor that drives accessibility in fiber blasts. Now what you can do is take a random motif like OXOX. This is the OXOX motif. This is of a transcription factor that is not expressed at all in FibroBlast. And we can do an experiment where we just insert this motif into this enhancer. It should do nothing because OXOX is not expressed. Watch what happens. As we insert motifs, look at the prediction scores. They're starting to drop dramatically. And if you zoom in, you see that the model has learned OXOX as a repressor. As you add these OXOX motifs, the model has started to essentially repress activity in a cell type where the OXOX transcription factor isn't even expressed. If you take single-task models instead, a single model trained on fibroblast ataxic data, again, just small model to KB sequence, you do the same experiment, the model doesn't react at all to the motif. This is a pretty pervasive issue, by the way. We're having a paper coming out soon on this. All the big multitask models look very good on performance on observational data. When you do counterfactual experiments, you learn that there's massive leakage of features across tasks. They're learning representations that are entangled, and an activator in one cell type has a negative association with activity in another cell type. It ends up being learned as a fake repressor. If you don't interpret the models, you would never know. And if you wouldn't build really adversarial benchmarks like I just showed you, putting a motif of a



transcription factor that isn't even expressed in the sequence, you couldn't tell. And this has real consequences because if you look at sequence design applications where often these models are used to design enhancers, promoters, you can see these effects. Like here we are trying to design sequences that are specifically active in keratinocytes versus melanocytes. The model can achieve this very well. Like you can hit the target performance and the target score. Everything looks great. So when you look at the predictions of the model, it looks like your sequences are behaving as expected. When you interpret them, you realize they're using only repressors mostly to get the differential effects. And when you look at what they're using, it's often TFs that aren't even expressed. So these are dangers of benchmarking models only on performance without knowing what exactly they're doing. This is also dangerous of using big models that throw everything in as a soup. You can end up with all this kind of weird confounding going on. So I'll stop there and just summarize. The highlights I want to really focus on is on exhaustive model interpretations actually really important, not just to reveal the biological understanding, which is nice, but also to understand the artifacts of the data and the artifacts of the model. Like for example, enzyme biases or spurious non-causal features like I showed. Sequence-to-function models can learn causality from observational data, but not in all cases. I showed you the example of the MITA task model, which ends up learning non-causal associations, despite the fact that in principle, sequence-to-function models can learn causality. Evaluation on counterfactual prediction tasks is very important because Enfirma, for example, consistently outperforms Chrome VP NAT on observational cross-validation, but is not able to beat it on counterfactual effects. Data and model scaling alone is not the optimal strategy necessarily. You really need to think about how you're training the model, not just what you're feeding it. And finally, improving data quality will often beat adding more parameters and more data. And with that, I'll just thank my lab and all the collaborators. Thank you. Thanks, Abdul. That's a really great talk. Just to remind everyone, now that you have the press to go into the URL, we'll give you a second question that we're going to come to take later. It's a combination of multiple things. Certainly base pair resolution helps. For example, alpha genome does much better than FOMO. It is in fact base pair resolution. That makes the model much more stable. Because as you increase resolution, for example, there were models that predicted the scalar outputs. You can go, there are many ways to go from sequence to a single scalar output. And different iterations of the model will learn different syntax, right? And you look at base pair resolution, the footprints and so forth really align very nicely with the syntax. So you end up learning much more stable models. So stability helps there. But what's really happening is a training strategy. So when you train these big long context models, the number of base pairs that are actually showing activity in those long context windows is pretty small. We play with upsampling data. So for example, since our sequences are short, we can upsample regions with signal, like peaks, in training, compared to background. You know, training in long context models, you're stuck with the long sequence you have, and you don't have a good

way of actually sampling regions within that sequence. So by simply tuning the training strategy, we can train the same models to just randomly sampling the genome. They do much worse. So you can play with training strategies, do bootstrap resampling, play with loss functions, combine multiple metrics, ensemble models, all of those will give you big boosts, which you may or may not be able to do with larger models. Yes. For a community which are based on which are the key criteria that such baseline models should go? I think it's very domain specific. It's going to be very hard to say what they should be. I think they should be models that are state-of-the-art in the community and are being used frequently for downstream applications. The applications are really bad. Cross-validation is not a valid application. The problem with many of these DNA language models, for example, is in all their benchmarks, they're simply looking at cross-validation performance of observational data. Like can I predict gene expression? Can I predict where the peaks are in the genome? But the actual applications of these models are variant effect prediction, sequence design, interpretation. So your tasks itself have to map to what people really care about downstream, not the surrogate tasks that you use to train the models. For the baselines, again, it really depends on the domain. But I guess a trend is to train larger and larger models. I think people need to push small models to the limit to see how much further you can go versus the large models that you have. Tend to bind. I wonder if you can get rid of the. So these proteins typically tend to recognize and bind sequence-specific motifs. And they bind in complex configurations. So there is not just an additive effect of things binding together, but there's often complex or combinatorial synergistic effect of proteins binding DNA. And understanding this kind of cis-regulatory code, that is a sequence syntax of these elements, is a pretty fundamental question in gene regulation. Now, the nice thing is you can actually use a variety of different profiling assays to capture different biochemical markers that represent different kinds of regulatory activity. For example, this track here is showing a ChIP-seq experiment capturing binding of this black triangular protein at specific locations. Similarly, you have various kinds of epigenetic or chromatin modifications. Those can also be captured by various readouts, and you can, of course, measure DNA methylation, gene expression, and so forth. So we can start using large-scale genome profiling in a variety of different cell types and tissues to create these reference regulomes. Just as we have the reference genome, we now have large numbers of assays performed across the entire genome in hundreds and thousands of cell types, tissues, cell lines, and now at single cell resolution. And one important point here is that a lot of this activity of the genome is highly context-specific. So here's a piece of the genome, one particular gene. These are all a variety of different immune cell types. We're looking at chromatin accessibility, which is sort of a measure of open chromatin, where proteins typically bind in the genome. And you can see, you know, like different parts of the genome, like this promoter idea is open in specific cell types and not open in others. Others are open in other cell types. So context really matters. And that's a really important piece of this puzzle as well. So I'll start off with models that we've built in the lab and literally the

opposite of a foundation model. These are models trained on individual experiments. So you performed a ChIP-seq experiment or a DNA's or a tax-seek experiment in some cell type. Your goal is to precisely capture that model in the context of how DNA sequence regulates that particular assay's activity. So it's a model per experiment. It's a sequence-to-function model, which means it takes DNA sequence as the input and maps it to base-pair resolution profiles, coverage profiles of an experiment. And these are very simple models. They only take about two kilobases of sequence at a time. So they are short receptor fields. But they're really modeling data at single base pair resolution. That's a key aspect. And we've built a variety of these models. We first published BPNet in 2021. This was focused on transcription factor binding, so protein DNA interactions for chip-exo, chip-seq, cut and run-like assays. Then we released Chrome BPNet, which is sort of a version of this for attack-seq, DNA-seq, single-cell attack-seq. We've also recently released ProCapNet, which is a similar model for nascent and steady-state transcription start site activity. So that's for ProCap, cage, and rampage-like assays. And we've been expanding across the assay space. So we have a specialized model for every assay, kind of the exact opposite of a classical foundation model. And the reason we did it this way is because we felt like you can go pretty far, firstly, with small models. You have more control of how you train them, and we can interpret them at scale very robustly to understand what they're learning, thereby providing very strong baselines for much larger models. That's the idea behind it. So one of the first things we can do with a trained model, if assuming it's accurate, is, for example, we can take, I'm sorry, you can't see this very well with the stuff on top. What you're seeing here is actually sequence, and the model has been already trained to predict profiles. We're able to reverse engineer what the model is learning for a specific sequence of interest and get these contribution scores. Every base pair in the sequence is given a score. That score highlights the contribution of that base pair to the predicted activity from the model. So it allows you to exactly identify base pairs, motifs, and other features in the sequence that are likely driving activity. And we've developed a variety of these kinds of approaches. I'll walk you through a few examples of how this works. So here's the MIC promoter in the human genome. There are two transcription start sites. We're looking at a Luke-Mes aligned K562. Here's the experiment. It's a pro-CAP experiment. It gives you base pair resolution precision of where transcription start sites are, and it measures nascent transcription. You train a model, this kind of small pro-CAP net model on that data. It really beautifully captures the data very well. So this is a cross-validation chromosome. So this sequence has not been seen in training. You can see the model very nicely recapitulates what was seen in the data. The question that you might want to ask is like, how is the model making this prediction? That's the interesting question. Because you already have the data. There's much value in predicting data you already have, unless the model is actually learning some very exciting biology. So we can take the model, we can apply these post hoc attribution methods that take the model as a lens, and you can now take these base pairs, which when you just look at them naively, you might not know what they do, and use a model to figure

out which base pairs are interesting. So when you do that, you get this contribution scope profile, which says most base pairs in the sequence are not doing much. But right there, there is a really interesting syntax that pops up. At this main transcription side of MIC, there's a classic TAINR. It's initiator motif. The model picks it up very nicely. There's a cutter box exactly 30 base pairs upstream and an SPBRE motif further upstream. For this other TSS right here, again, you see the cutter box and the SPI motif, but watch this one, right? You see how the model focuses on a single A. It says that there's a single A in the genome that allows you to initialize transcription. Obviously, there are a million A's or billion A's inside the human genome. Every A doesn't initiate transcription. The reason this happens is because this A is exactly 30-base pairs downstream of a tarot box, right? And I don't have time to show you this, but you can do ablation experiments where you can mutate one motif, mutate another one, and see the effect on the activity. And you can clearly see that if you mutate, for example, or you have a mutation in the terra box, this INR disappears. So you can see how the model beautifully learns epistasis. But also shows you the ability of using the model to get really beautiful, clean insights into the actual sequence code that drives transcription. Now let's say we switch to a different assay. So it's the same cell type. It's the same promoter. But this time we switched to DNA-seq, which measures chromatin accessibility. It's the exact same sequence, the same cell type. Now we take a different model. This one is called Chrome EPNet, not ProCapNet. This one is trained on DNA's data. And you can see the model very nicely predicts the data as seen at base-pair resolution. I'll talk about bias correction in a bit, but one very important thing to realize is, and I'm sure we all know this, but often don't really process it, is that the data that you see from an experiment is not the ground truth. It is a ground truth confounded by various technical artifacts. So if you train a very powerful model on confounded data, your model is going to learn biology and confounding. That's what we see with many of these assays. For example, Chrome BPNAT, if you train any sequence-to-function model, if you train it on Attack-Seq or DNA-seq, it hyper-focuses not only on the regulatory biology, but also on the enzyme biases. And I'll show you the massive impact that these can have on what you see from the experiments. But anyway, the model predicts it very well. We can do bias correction. Just take my word for it right now. And now you take the same promoter and interpret it through the lens of chromatin accessibility, right, not nascent transcription. You see a very different code pop up. It's the same sequence. The contribution scores are very different. These INR starter boxes and so forth don't pop up. But you see these kind of GC-rich sequences pop up. And there is a GATA motif. It's the reverse complement of GADA. And you can see how it specifically pops up in the chromatin accessibility track, and it's hugging this particular initiator. So you have different parts of the sequence being utilized by different biochemical processes. You have the initiator, the tarot box really positioning transcription, but the chromatin accessibility is regulated by a completely different syntax in the exact same sequence. And the model can tell you that. You can decompose this further, because luckily in this cell line, we have ChIP-seq data for multiple

transcription factors. So we can also train models on the ChIP-seq data itself, one model for every transcription factor, and interpret the same sequence through the lens of those models. And you see that the SP1 ChIP-seq, right, the model that's trained on that, its contribution scores almost exactly mimic the chromatin accessibility contribution scores, except for the GATA motif. Because SP1 does not recognize GATDA. It can recognize these GC-rich-like sequences. But if you look at a GATA chip-seq model trained in the same cell line, it precisely picks up the GATA motif, right? So you can now say that the chromatin accessibility track that you observed over MIC is essentially a combination of GATA and SP1 collaborating to open chromatin. Thereafter, you have initiation, which is positioned by these specific initiatives. Now let's switch the context again. This time we're going to keep the MIC promoter as it is. We're going to keep the assay again DNA-seq. We're going to switch the cell type. We're going to go from a K562 leukemia cell line to a HepG2 liver carcinoma cell line. And if you look at the data, right, just look at the observed data, it looks almost identical. You can't really make out the difference between the two cell lines, at least base by resolution. The model predicts it almost identically as well, does bias correction. It's very unfortunate that it's hiding the last track. But luckily, it's not hidden the most important piece, which is, take my word for it, that the contribution scores from the HepG2 model look exactly like the K562 model, except at this position. You see this GATA motif right here? It's gone, right? It's one motif, that's about it. And the model has never seen gene expression. It's just training on chromatin accessibility using sequence only. The GATA transcription factor is not expressed in HepG2. So the model has somehow learned, not magically, but looking at other sequences that are accessible, that despite the profile looking nearly identical, the GATA motif is not actually regulating accessibility in the liver saline. And what effect does that have? Chromatin accessibility is significantly lower in HEPG2 than K562. So it affects the magnitude of accessibility, but not the profile shape. So there are different axes to these assays as well. We can go to an enhancer. This is an enhancer of MIC, validated. And we're going to look at DNAs. Again, observed, predicted. And this is the bias-corrected DNAs. We have a method to actually correct for the enzyme bias, which we'll use model interpretation itself to learn. We can interpret this enhancer, but let's look at ATTC-Seq. So these, again, DNA-seq and ATT-Seq are supposed to be assays capturing the same phenomenon of code and accessibility. But when you look at them at base-pair resolution, they look remarkably different. They literally don't look like the same thing at all, right? Despite they're supposed to capture the same thing. If you naively train a machine learning model on either of these assays, it beautifully mimics the assay itself. Look at our predictions. Doesn't it look exactly like ATT ?? CK-Seq, and this one looks exactly like DNA-seq. But now if you correct the enzyme biases, which again, you have a neural network learn the enzyme biases from the data itself, you can regress it out and look at the underlying latent, hopefully ground truth. And what you see for ATT ?? CK is after you correct for TN5 enzyme biases, you get this latent profile, which is very similar to the DNA's one. So the observed data that you got from the

ATTAC-seq data was actually corrupted by this TN5 bias. It is not reality. It's not ground truth. It is a ground truth corrupted by a heavy enzyme bias. DNA's has a much less enzyme bias. So even after bias correction, the predictions look much more similar to the observed data. Now the cool thing is you can interpret both models. Again, two separate models trained on two different assays independently, never talking to each other. And you can ask the models, hey, tell me what syntax is driving activity accessibility in this enhancer. And you can see the two models give you nearly identical syntax. So after correcting biases, you would really make them look nearly identical, not only in the predicted space, but in the most important space, which is actually the features being used to make predictions. And then you can take models trained on a variety of different other assays to support what you see and to understand the context specificity. So if you train a model on an MPRA, it gives you similar syntax, but not identical. For example, this motif here is specifically found driving activation, but not opening chromatin. So you can really start now dissecting the precise roles of individual base pairs and motifs and driving different biochemical processes through the process of transcription. Now these are all cool stories, but you need validation. So this is after all a benchmarking talk. So what we did here is we collaborated with Axel Whistle, Len Panakio and Michael Koziki, and they did this insane experiment where they took a handful of enhancers and they put these in mouse embryos. And these are in vivo reporters. So you're taking these enhancers, you're taking the sequences, you're mutating 12 base map blocks at a time, and they do random scrambles. So you don't know what exactly they're mutating. And you can put these into the embryo and then different parts of the embryo light up. So at the same time, you can figure out which cell type, for example, the enhancer is having an effect and what its effect size is. It's not super quantitative, but it tells you at least binary, you know, does it have an effect, does it have a strong effect, weak effect, and you can see where in the mouse embryo it has an effect, at what stage of development. We can use this as a ground tool to try to test whether models trained, this time, our models are trained on chromatin accessibility in a large number of human cell types and cell cell states. We're going to test them against a report of gene expression in a mouse. These human models trained on accessibility. It's a very hard generalization problem. We get very nice predictive power. So our models are able to get 88% specificity and 60% sensitivity. Now this is not 100%. It should not be, because we are switching assay and we're switching species. But the most exciting piece is we can look at these blocks that cause effects and we can actually use the model to interpret what is causing the effect. So we can tell, for example, this gain of function effect is because the scramble destroyed a SNI repressor. This loss of function effect is because you scramble SOX motif. So you can get really nice explanatory power to do this as well. Another interesting application of this is to use this for variant effect prediction. So you can take any genetic variant, you can take a trained model, you can insert the reference allele, alternate allele, and make predictions, but also actually interpret the model and make interesting insights. For example, this C reference allele doesn't do much. There's a weak

motif right here. And the G allele creates a really strong SPI motif that interacts with this hidden SPI motif. And they collaborate to create a massive amplification of the accessibility at that location. So you're again a nice way of interpreting genetic variants. We can do this at scale. So here we're trying to again use a benchmarking data set. Here what we're going to do is train a reference model on a lymphoblastoid saline from a European individual. It's a single model trained on the reference genome in a reference individual. We're going to use it to try to predict QTLs. These are genetic variants that have associations with changes in chromatin accessibility across a large panel of 100 diverse African individuals. So we're switching ancestry and we're using this QTL data as a ground truth. We train our model on a reference individual and then we'll make predictions for all these variants. On the x-axis is the actual observed data, the effect sizes. On the y-axis are the predicted. And you can see it's not perfect, but there's very nice sign concordance, about 0.69 correlation. This one model trained on one data set on a reference individual. Now let's switch to the foundation models, right? The ones that are trained on large number of individuals, large number of data sets, simultaneously in multitask mode. And we'll look at Informa, which was previous state of the art, this new alpha genome that just came out a few weeks ago. On the y-axis here is again the correlation between predicted and observed effect sizes. The blue triangles you see here are Chrome VVNet. That's a small model. On the x-axis, which you cannot see, are various thresholds for how you call significant QTLs. So we're changing the threshold just to be threshold independent. And we're going to see how performance of these models changes as a function of threshold. You can see that they perform pretty uniformly across threshold, which is nice. But you'll see that these yellow points are using Enfirma as published in the original paper. And they perform really quite poorly. Part of the reason is because they have a weird strategy of predicting variant effects, which is they compute the effect of the variant on a 100 kilobase window of profile. So even though the variant has a local effect on signal, it gets washed out when you look at it 100 kilobases. If you fix that kind of bug, they do quite a bit better. That's the green points. But you can see the RL models actually outperform it by a decent margin. You can switch this prediction task to discriminating between significant QTLs and background QTLs, not just predicting effect size. The y-axis here is now average precision. Again, you're discriminating between variants that have effects and variants that don't. You can again see this is the baseline performance of the random classifier. This is NFOMA as published. This is NFOMA after correcting the variant effect bias. This is our Chrome BVNET models. What's really interesting is we are very much on par with them. But as soon as you increase the sequencing depth of the assay, not the size of the model, not the number of parameters, not the amount of data, just the reads, the number of reads, the quality of the data, you see this giant boost in performance. So data quality, improving that often will boost your model better than simply increasing the size of your model or the number of parameters or the amount of data you have. And just to be clear, Enformer has two orders of magnitude more parameters than ours. It also is trained on 6,000 or 10,000

tracks simultaneously. Chromebyner is trained on a single track. It only has a 2KB receptor field versus Enformer's 100 kilobase receptor field. So despite having all the necessary advantages, capacity, model capacity, size, it ends up doing comparably or slightly worse than what we're able to do. Let's move to another class of foundation models, which is the self-supervised foundation models. These have been very big as well. They're often called DNA language models or regulatory language models. I'm going to focus here on what I'm referring to as annotation agnostic models. So these are models that are trained on entire genomes across multiple species in a self-supervised way to reconstruct sequences that are used as inputs. You just mask or you do autoregression. And the point is that they're supposed to be able to learn powerful representations, which you can then use for fine-tuning or probing for other supervised tasks. So we're looking at a few of the state-of-the-art models, and Evo2 can also be added to this bar. We're looking at DNA BERT, GNGen LM, Hyena DNA, nuclear transformer. We're going to test these models in the same variant prediction task that I showed you in the previous slide. Chrome BPNET, can it predict these covenant QTLs? And just for reference, AUROC 0.5 is random. Chrome BPNET's performance on two separate data sets is about 0.75 year and about 0.93R. So that's the baseline. The baseline is not a random predictor. The baseline is not another language model. The baseline is what should be the state-of-the-art small model that can get you the best performance. That's how you should be benchmarking. The trick that's used in all the papers is we compare to the previous iteration of what someone else did in the same space and see if it does better. Now, the point is that in ZeroShot, all these models are basically random. They cannot predict these variants at all. You can probe them, which is you can add a linear head on top of the last layer of the model. They just barely cross random. Even if you fine-tune the whole model on the exact data that Chrome VPNet has access to, they're still substantially worse, right? So they're nowhere close. And you can add other DN language models here too. It's the same story. So just because you have large parameter space and a lot of data doesn't necessarily mean your model is actually going to do well. In fact, a lot of these DN language models that are trained often do very poorly on counterfactual prediction. Even if they're fine-tuned, they end up doing pretty poorly on these, especially on the non-coding genome. They're pretty good on coding sequence. But non-coding sequence, they really aren't. We have various other benchmarks too. Here we're looking at effects on the, these are in-genome effects with CRISPR experiments on the promoter of a gene called PPIF. Again, Chrome-BPNAT, even though it's trained on chromatin accessibility, is able to predict effects on expression with like 0.65 correlation. If you take Enforma's expression head and you make predictions, it gets a correlation of 0.42. If you take Nforma's accessibility head and predict expression effects, it gets 0.62. So again, the small model is able to outperform the big one. On this particular task, Chrome BPNED also outperforms AlphaGenome, the latest iteration of this kind of models. You can go to the MPRA benchmark. So this is the KG5 competition that was held a few years ago. These are report assays where you do saturated mutagenesis of promoters and enhancers. Each



point here is a promoter or enhancer of a specific gene. That's the name you're seeing. On the y-axis is the correlation of Enformer's predictions against the ground truth. On the x-axis is the predictions of Chrome BPNET against the ground truth. The diagonal would be, they're equally performing. And if you're on the bottom of the diagonal, Chrome BPNET outperforms Enformer. On the top of the diagonal, Enforma would outperform Chrome BPNET. You can see right here, they're pretty much on par. And on some examples, we do substantially better. So overall, the key point is that when you're benchmarking these models, you need to kind of really think about what's your actual baseline, right? Five minutes. It cannot be, again, like I said, a random classifier or something like that. You don't have to push your baseline models to the limit. In many cases, these small models trained on small amounts of data in specific ways can take you pretty far, besides having the advantages of interpretability. One last thing I want to show is that these large multitask models, like Informa, Deep C, Alpha Genome, again, they take sequences input and they're used to predict thousands of tracks simultaneously. There's another problem that occurs is since they are learning joint representations, the causality of these models starts coming into question. I'll show you a cool example of this. So here we're taking the deep C model, which is trained on, again, thousands of tracks. We're taking the reference sequence and we're interpreting it. You can see right here, there's a bunch of AP1 motifs that drive activity. This makes sense. AP1 is a primary transcription factor that drives accessibility in Fiber Blus. Now what you can do is take a random motif like OXOX. This is the OXOX motif. This is of a transcription factor that is not expressed at all in FibroBlus. And we can do an experiment where we just insert this motif into this enhancer. It should do nothing because OXOX is not expressed. Watch what happens. As we insert motifs, look at the prediction scores. They're starting to drop dramatically. And if you zoom in, you see that the model has learned OXOX as a repressor. As you add these OXOX motifs, the model has started to essentially repress activity in a cell type where the OXOX transcription factor isn't even expressed. If you take single-task models instead, a single model trained on fibroblast ataxic data, again, just small model to KB sequence, you do the same experiment, the model doesn't react at all to the motif. This is a pretty pervasive issue, by the way. We're having a paper coming out soon on this. All the big multitask models look very good on performance on observational data. When you do counterfactual experiments, you learn that there's massive leakage of features across tasks. They're learning representations that are entangled, and an activator in one cell type has a negative association with activity in another cell type. It ends up being learned as a fake repressor. If you don't interpret the models, you'd never know. And if you wouldn't build really adversarial benchmarks like I just showed you, putting a motif of a transcription factor that isn't even expressed in the sequence, you couldn't tell. And this has real consequences, because if you look at sequence design applications, where often these models are used to design enhancers, promoters, you can see these effects. Like here we are trying to design sequences that are specifically active in keratinocytes versus melanocytes. The model can

achieve this very well. Like you can hit the target performance and the target score. Everything looks great. So when you look at the predictions of the model, it looks like your sequences are behaving as expected. When you interpret them, you realize they are using only repressors mostly to get the differential effects. And when you look at what they're using, it's often TFs that aren't even expressed. So this is the dangers of benchmarking models only on performance without knowing what exactly they're doing. This is also dangerous of using big models that throw everything in as a soup. You can end up with all this kind of weird confounding going on. So I'll stop there and just summarize. The highlights I want to really focus on is on exhaustive model interpretations actually really important, not just to reveal the biological understanding, which is nice, but also to understand the artifacts of the data and the artifacts of the model. Like for example, enzyme biases or spurious non-causal features like I showed. Sequence-to-function models can learn causality from observational data, but not in all cases. I showed you the example in the MADA task model, which ends up learning non-causal associations, despite the fact that in principle, sequence-to-functional models can learn causality. Evaluation on counterfactual prediction tasks is very important because Enfirma, for example, consistently outperforms Chrome VPNA on observational cross-validation, but is not able to beat it on counterfactual effects. Data and model scaling alone is not the optimal strategy necessarily. You really need to think about how you're training the model, not just what you're feeding it. And finally, improving data quality will often beat adding more parameters and more data. And with that, I'll just thank my lab and all the collaborators. Thank you. Thanks, Abu. That's a really great talk. Just to remind everyone, you're going to get you and thank you for the content later. But I think that's a very important thing. It's a combination of multiple things. Certainly, base pair resolution helps. For example, Alpha Genome does much better than Former. It is in fact base pair resolution. That makes the model much more stable. Because as you increase resolution, for example, there were models that predicted the scalar outputs. You can go, there are many ways to go from sequence to a single scalar output. And different iterations of the model will learn different syntax, right? And you look at base pair resolution, the footprints and so forth really align very nicely with the syntax. So you end up learning much more stable models. So stability helps there. But what's really happening is a training strategy. So when you train these big long context models, the number of base pairs that are actually showing activity in those long context windows is pretty small. We play with upsampling data. So for example, since our sequences are short, we can upsample regions with signal, like peaks, in training, compared to background. You know, training in long context models, you're stuck with the long sequence you have, and you don't have a good way of actually sampling regions within that sequence. So by simply tuning the training strategy, we can train the same models to just randomly sampling the genome. They do much worse. So you can play with training strategies, do baseball resolution, play with loss functions, combine multiple metrics, ensemble models, all of those will give you a big boost, which you may or may not be able to

do with larger models. Yes. Which are the key criteria that such baseline models should go? I think it's very domain specific. It's going to be very hard to say what they should be. I think they should be models that are state-of-the-art in the community and are being used frequently for downstream applications. The applications are really bad. Cross-validation is not a valid application. The problem with many of these DNA language models, for example, is in all their benchmarks, they're simply looking at cross-validation performance of observational data. Like, can I predict gene expression? Can I predict where the peaks are in the genome? But the actual application of these models are variant-defect prediction, sequence design, interpretation. So your tasks itself have to map to what people really care about downstream, not the surrogate tasks that you use to train the models. For the baselines, again, it really depends on the domain. But I guess a trend is to train larger and larger models. I think people need to push small models to the limit to see how much further you can go versus the large models that you have.